BOS-OMEGA

Bioweapon Orchestration System — Omega Class

AI Orchestration Platform · Full Technical Reference · CLASSIFIED

CLASSIFIED
UMBRELLA CORP
May 2026

This document is the complete technical reference for BOS-OMEGA: its architecture, API surface, database schema, orchestration pipeline logic, frontend design, and operational configuration. All access is logged and subject to §13.7 enforcement.

1. WHAT IT IS

BOS-OMEGA is a production AI orchestration platform styled after Umbrella Corporation's Resident Evil universe. It accepts a free-text task, classifies it, selects the optimal execution strategy (single / parallel / consensus / series_pass / boil_the_ocean), routes across one or more LLM providers with automatic failover, validates output, and returns a structured GO / HOLD / ABORT tri-state decision with full explanation.

It is a real production system — not a demo — with full auth, multi-tenant per-layer memory, image generation with daily spend caps, audit logging, circuit breakers, provider health tracking, conversation continuity, and a complete admin surface.

2. MONOREPO LAYOUT

Managed with pnpm workspaces. A shared reverse proxy routes by path prefix: /api !' api-server, / !' bos-omega. Services bind to $PORT — never hardcoded. OpenAPI is the single source of truth; client hooks and Zod schemas are generated from it.

```
artifacts/
  api-server/          Express API + BOS orchestration engine  (TypeScript)
  bos-omega/           React 19 + Vite frontend  (TypeScript)
  mockup-sandbox/      Vite component preview server  (canvas only)
lib/
  db/                  Drizzle ORM schema + pg client  (@workspace/db)
  api-spec/            Single OpenAPI YAML source of truth  (openapi.yaml)
  api-zod/             Orval-generated Zod validation schemas
  api-client-react/    Orval-generated TanStack Query v5 React hooks
  local-agent-contracts/   Types for local-agent subsystem
  local-agent-policy/      Agent policy definitions
scripts/               Utility / maintenance scripts  (@workspace/scripts)
```

3. BACKEND — artifacts/api-server

Stack: Express.js · TypeScript · Drizzle ORM · PostgreSQL · pino structured logging · express-session · zod. Entry: src/index.ts !' bootstrap() seeds super-admin + default providers !' listens on $PORT. Auth roles: user / admin / super_admin.

API Routes (all prefixed /api)

Route	Purpose
POST /auth/signup GET|POST /auth/login POST /auth/logout GET /auth/me	Session-cookie auth. Signup with role. Login returns session cookie. Whoami.
POST /tasks	Core endpoint. Runs the full BOS pipeline. Stays open until complete.
GET /tasks GET /tasks/:id	List tasks (with filters) · Task detail including full BosOutput JSONB.
GET /tasks/:id/memory-context	Returns the exact memory string injected for that task run.
GET /tasks/:id/attempts /tasks/:id/runs	Per-model attempt records and execution run records for a task.
GET /runs/:id/series /runs/:id/parallel-agents /runs/:id/synthesis	Series-pass records · Parallel agent records · Consensus synthesis report.
GET|POST|PATCH|DELETE /providers	LLM provider CRUD. Enables/disables provider, sets priority.
PUT /providers/:id/api-key DELETE /providers/:id/api-key	Store / clear AES-256-GCM encrypted API key. Never returned in plaintext.
POST /providers/:id/test	Live credential validation (rate-limited). Returns latency + model list.
POST /providers/:id/discover-models	Auto-fetch + register model catalog from provider API (rate-limited).
GET /providers/preflight	"Is any reachable key configured?" quick liveness check.
GET /models PATCH /models/:slot	Model registry read / slot update (capability tags, scores, enable toggle).
GET|PATCH /personas	3 persona slots (A/B/C) — editable system-prompt overlays.
GET|PUT|DELETE /memory/budgets	Per-user per-layer token budget overrides (canon/continuity/patches/scratchpad).
GET|POST|DELETE /scratchpad POST /scratchpad/pin	Per-user continuity scratchpad. Pin an assistant message as a note.
GET /lattice/export POST /lattice/import GET /lattice/exports	Full continuity bundle export/import as JSON. Last-10 snapshot list.
GET|PATCH /conversations	List conversations · Rename conversation.
GET /fallbacks GET /audit	Provider failover event log · Compliance audit log.
GET|POST|PATCH /users POST /users/owner/repair	User management (admin+ only). Re-seed super-admin from bootstrap password.
GET /image-quota PUT /overrides/users/:id/override	User's daily image-gen cap + usage · Per-user quota override (admin only).
GET|POST|DELETE /uploads	File upload, list, raw bytes, thumbnail, delete.
GET /health GET /tri-state	Liveness check · Aggregate GO/HOLD/ABORT stats + latency + success rate.

4. THE BOS PIPELINE (src/bos/pipeline.ts · ~40 modules · ~10,000 lines total)

POST /api/tasks runs the full pipeline synchronously — the HTTP request stays open until the pipeline finishes. The pipeline is the core product. It is purely in-process; no queues or background workers.

```
Input
  |-- Input Gate              Content-safety pre-filter. Returns ABORT on blocked inputs.
  |-- Front Door Interpreter  Classifies conversational intent BEFORE touching the LLM budget.
  |     Outcomes: GREETING | EMPTY | UNDER_SPECIFIED | LIKELY_NON_TASK
  |     If matched -> return guidance + example prompts (zero LLM spend)
  |-- Task Classifier         Maps free-text -> task_type:
  |     legal | code | math | research | summarization |
  |     extraction | planning | creative | safety_review | general
  |-- Image Intent Detector   Detects image-gen or image-edit intent ->
  |     routes to imageProviderBridge.ts (DALL-E / compatible bridge)
  |-- Memory Engine           Loads 4 layers with per-user token budgets:
  |     canon        -- immutable governance / behaviour contract
  |     continuity   -- carry-over notes from prior tasks
  |     patches      -- targeted one-off overrides of canon rules
  |     scratchpad   -- short-lived working notes for this session
  |-- Persona Overlay         Injects slot A/B/C persona system-prompt if requested
  |-- Mode Selector           Chooses ExecutionMode via heuristics (see table below)
  |-- Model Router            Scores all enabled models -> weighted ranking
  |     Weights: capability_match x3.0  reliability x2.0  health  latency  cost
  |-- Execution Engine        Dispatches by selected mode:
  |     single          -> 1 model call
  |     parallel        -> N concurrent calls -> pick highest confidence score
  |     consensus       -> N calls -> ConsensusMerge -> select + rationale
  |     series_pass     -> draft -> LLM critique -> LLM refine (3-pass minimum)
  |     boil_the_ocean  -> exhaustive multi-pass for high-stakes tasks
  |-- Validation Engine       schema_pass * safety_pass * instruction_pass * completeness_pass
  |-- Repair Engine           If validation fails: attempt structured repair, re-validate
  |-- Tri-State Decision      GO | HOLD | ABORT + explanation + safe_alternative
  |-- Audit Engine            Write event + sha256(input) to audit_logs table
  |-- Scratchpad Writer       Auto-summary entry written post-task (async, non-blocking)
  |-- Circuit Breaker         Update providerHealthTable on success / failure
Output: BosOutput { state, answer, assumptions, uncertainties, failure_modes,
                    recommended_next_action, why_decision_was_made,
                    safe_alternative, parallel_responses, generated_attachments }
```

Mode Selector Heuristics (when mode = "auto")

Trigger Condition	Selected Mode
"exhaustive" / "comprehensive" / "production ready" / "bulletproof" / "final version" in input	boil_the_ocean
"refine" / "improve" / "check my work" / "polish" / "iterate" / "double check" in input	series_pass
High-stakes task type (legal / research / planning / code) AND input > 200 chars	boil_the_ocean
Complex task type AND input > 100 chars	series_pass
Any input > 500 chars	series_pass
Everything else	normal (single-model)

Core TypeScript Types (src/bos/types.ts)

```typescript
type TriState       = "GO" | "HOLD" | "ABORT"
type ExecutionMode  = "single"|"parallel"|"consensus"|"series_pass"|"boil_the_ocean"|"auto"
type TaskType       = "legal"|"code"|"math"|"research"|"summarization"|
                      "extraction"|"planning"|"creative"|"safety_review"|"general"
type ProviderStatus = "HEALTHY" | "DEGRADED" | "OPEN_CIRCUIT" | "RECOVERY_TEST"

interface BosOutput {
  state: TriState;  task_type: string;  answer: string
  assumptions: string[];  uncertainties: string[];  missing_inputs: string[]
  failure_modes: string[];  recommended_next_action: string
  why_decision_was_made?: string      // HOLD / ABORT only
  safe_alternative?: string           // HOLD / ABORT only
  front_door_route?: string           // GREETING|EMPTY|UNDER_SPECIFIED|LIKELY_NON_TASK
  front_door_examples?: string[]
  parallel_responses?: ParallelResponse[]
  generated_attachments?: GeneratedImageRef[]
  repair_applied?: boolean
}
interface TaskContext {
  task_id: string;  input: string;  task_type: string
  tri_state: TriState;  mode: ExecutionMode;  parallel_models: number
  memory_context?: string         // pre-rendered once, shared by ALL engine modes
  persona_prompt_text?: string;  attachment_context?: string
  attachment_images?: VisionImage[]
}
```

5. DATABASE (lib/db) — Drizzle ORM · PostgreSQL · 30 Tables

Table	Key Columns / Purpose
users	id, email, password_hash, role (user|admin|super_admin), created_at
llm_providers	id, name, base_url, priority, enabled, encrypted_api_key (AES-256-GCM)
llm_models	id, provider_id, model_name, capability_tags[], context_window, cost_input/output, latency_score, reliability_score, enabled
provider_health	provider_id, status (HEALTHY|DEGRADED|OPEN_CIRCUIT|RECOVERY_TEST), failure_count, avg_latency_ms
tasks	id, user_id, conversation_id, input, output (BosOutput JSONB), state, task_type, mode, duration_ms, created_at
model_attempts	id, task_id, provider_id, model_id, latency_ms, token_input, token_output, cost_estimate, success
execution_runs	id, task_id, mode, selected_model, duration_ms, success
parallel_agents	id, run_id, provider, model, state, confidence_score, latency_ms, selected
series_passes	id, run_id, pass_number, raw_output, validation_passed
synthesis_reports	id, run_id, winner_agent_id, confidence, rationale
tri_state_decisions	id, task_id, state, cause, explanation, safe_alternative
fallback_events	id, task_id, from_provider_id, to_provider_id, reason, created_at
audit_logs	id, event_type, actor_id, task_id, input_sha256, details JSONB, created_at
memory_items	id, user_id, layer (canon|continuity|patches|scratchpad), title, content, authority_level, source, source_task_id
user_memory_budgets	user_id, canon, continuity, patches, scratchpad (per-layer token ceilings)
conversations	id, user_id, title, keywords[], created_at, updated_at
attachments	id, user_id, task_id, filename, mime, bytes (BYTEA), created_at
lattice_exports	id, user_id, fidelity_sha256, byte_size, task_count, created_at
image_quota_overrides	user_id, daily_count, daily_usd_cents, overridden_fields[]
validation_results	id, task_id, schema_pass, safety_pass, instruction_pass, completeness_pass, confidence_score
personas	slot (A|B|C), title, content, updated_at
bos_task_requests	Local agent task queue — id, org_id, device_id, payload JSONB, status
bos_task_executions	Local agent execution log — links to bos_task_requests
bos_org_* (4 tables)	Multi-org / device pairing, install modes, approval tokens (local agent subsystem)

6. CODE GENERATION (OpenAPI → Client)

```
lib/api-spec/openapi.yaml   <-- single source of truth for ALL API contracts
  |
  +-- pnpm --filter @workspace/api-spec run codegen
        |
        +-- lib/api-zod/          Zod schemas -- server uses these to validate req.body
        +-- lib/api-client-react/ TanStack Query v5 hooks -- frontend imports these

Examples of generated hooks:
  useListProviders()      useCreateProvider()     useUpdateProvider()
  useDeleteProvider()     useSetProviderApiKey()  useClearProviderApiKey()
  useTestProvider()       useDiscoverProviderModels()
  getListProvidersQueryKey()

Workflow when adding / changing an endpoint:
  1. Update lib/api-spec/openapi.yaml
  2. pnpm --filter @workspace/api-spec run codegen
  3. Update server route handler (validate req.body with generated Zod schema)
  4. Import generated hook in frontend component
```

7. FRONTEND — artifacts/bos-omega

Stack: React 19 · Vite · TypeScript · Tailwind CSS v4 · shadcn/ui · TanStack Query v5 · wouter (router) · framer-motion · lucide-react. All routes are auth-gated via session cookie. Unauthenticated → <Login />.

Pages

Route	Component	Purpose
/console	TaskConsole.tsx	Main chat UI. Submit tasks, view GO/HOLD/ABORT output, parallel responses, image attachments.
/memory	MemoryManager.tsx	Browse and edit the 4-layer memory system (canon / continuity / patches / scratchpad).
/providers	ProviderStatus.tsx	Ambient classified display — codenames (ORACLE-1, SENTINEL-A, APEX-G, SHADOW-L), signal bars, tier badges.
/users	Users.tsx	User management (super_admin only) — list, create, role change, password reset.
/settings	Settings.tsx	Appearance (11 themes), per-layer memory token budgets, scratchpad continuity viewer.

Navigation Structure

Sidebar width: w-52 (208px). Two labelled sections: Intel → [Console, Memory] | System → [Providers, Users*, Settings]. ConversationsList panel renders below the Intel section. (* super_admin only)

Key Components

Component	Purpose
Layout.tsx	Sidebar + top bar + content wrapper. Mounts MatrixRain canvas when theme = umbrella-corp.
MatrixRain.tsx	Animated canvas: red katakana + latin glyphs fall in columns. 7% of columns spell hidden messages in white. Sized via getBoundingClientRect() with window.innerWidth/Height fallback for production
ConversationsList.tsx	Collapsible conversation list in sidebar under the Intel section.
ProviderPreflightBanner.tsx	Warning banner when GET /api/providers/preflight reports no reachable API key.
MessageList.tsx	Per-message task output: Tri-State badge, answer, assumptions, uncertainties, image attachments, pin button.
CorporateLogo.tsx	Umbrella Corp SVG brand lockup (logo + wordmark variants, size prop).
StatusBadge.tsx	GO (green) / HOLD (amber) / ABORT (red) colour-coded pill badges.

Theming — 11 Skins

ID	Name	Visual Description
retro95	Windows 95	Pixelated bevels, MS Sans Serif, system gray — faithful Win95 aesthetic
retro98	Windows 98	Refined Win98 grays with title-bar gradient blue
modern	Modern	Warm-cream enterprise skin with rounded cards and muted tones
cyberdine	Cyberdyne	Terminator HUD: red glow on black, monospace terminal chrome
umbrella	Umbrella	Pitch black + Umbrella red, hex grid overlay, dark beveled cards
umbrella-corp	Umbrella Corp	Tactical enterprise console — matrix rain canvas, blinking threat indicators
capybara	Capybara	Cozy sage + cream pastels, heavily rounded, friendly warm aesthetic
anime	Anime	Hot pink + cyan + manga ink, chunky drop shadows, high contrast
steampunk	Steampunk	Parchment + brass + dark wood, serif typography, oxidized fixtures
neonpunk	Neon Punk	Cyberpunk grid: neon magenta + cyan on indigo-black
ultraclean	Ultra Clean	Pure white, jet black accents, hairline borders, zero decoration

Applied via CSS class on :root. Umbrella Corp styles scoped under :root.theme-umbrella-corp in index.css. Persisted to localStorage via useTheme(). Switchable live from /settings.

8. ENVIRONMENT VARIABLES & SECRETS

Variable	Service	Purpose
DATABASE_URL	api-server	PostgreSQL connection string (Replit managed DB)
SESSION_SECRET	api-server	express-session HMAC signing key — must be long and random
OWNER_SUPERADMIN_BOOTSTRAP_PASSWORD	api-server	Seeds the first super_admin row on cold start. POST /api/users/owner/repair uses this to re-seed
PORT	both	Injected per-service by Replit reverse proxy workflow. Never hardcode.
OPENAI_API_KEY	api-server	Optional env fallback if no encrypted DB key exists for the OpenAI provider row
ANTHROPIC_API_KEY	api-server	Optional env fallback for Anthropic
GEMINI_API_KEY	api-server	Optional env fallback for Google Gemini
OLLAMA_BASE_URL	api-server	Optional env fallback for Ollama (URL only, no API key required)
IMAGE_GENERATION_LIVE	api-server	Set to "true" for real image-gen API calls. Unset = mock stub mode.

All secrets managed via Replit Secrets UI — never committed to code or .env files. Provider API keys are stored in the DB encrypted with AES-256-GCM; the plaintext is never returned in any API response.

9. TESTING & TYPECHECKING

```
# Frontend unit tests (Vitest + React Testing Library) — 119 tests
pnpm --filter @workspace/bos-omega run test

# Full TypeScript check (builds composite libs, then leaf packages)
pnpm run typecheck

# Codegen only
pnpm --filter @workspace/api-spec run codegen

# Build artifacts
pnpm --filter @workspace/api-server run build
pnpm --filter @workspace/bos-omega run build

# Known pre-existing non-blocking TS warning:
#   MessageList.tsx references BosOutput.front_door_route — field exists
#   at runtime but was not reflected in openapi.yaml before codegen was last run.
#   Does not affect runtime behaviour.
```

10. DEPLOYMENT

Hosted on Replit. Each artifact binds to $PORT. The shared reverse proxy (configured via .replit-artifact/artifact.toml per service) routes by path prefix — most-specific-first, so /api never conflicts with /.

Published via Replit deployment system to a .replit.app domain (or custom domain if configured). PostgreSQL is Replit-managed. Secrets are managed via Replit Secrets UI. Two workflows run continuously: 'API Server' (pnpm --filter @workspace/api-server run dev) and 'web' (pnpm --filter @workspace/bos-omega run dev).